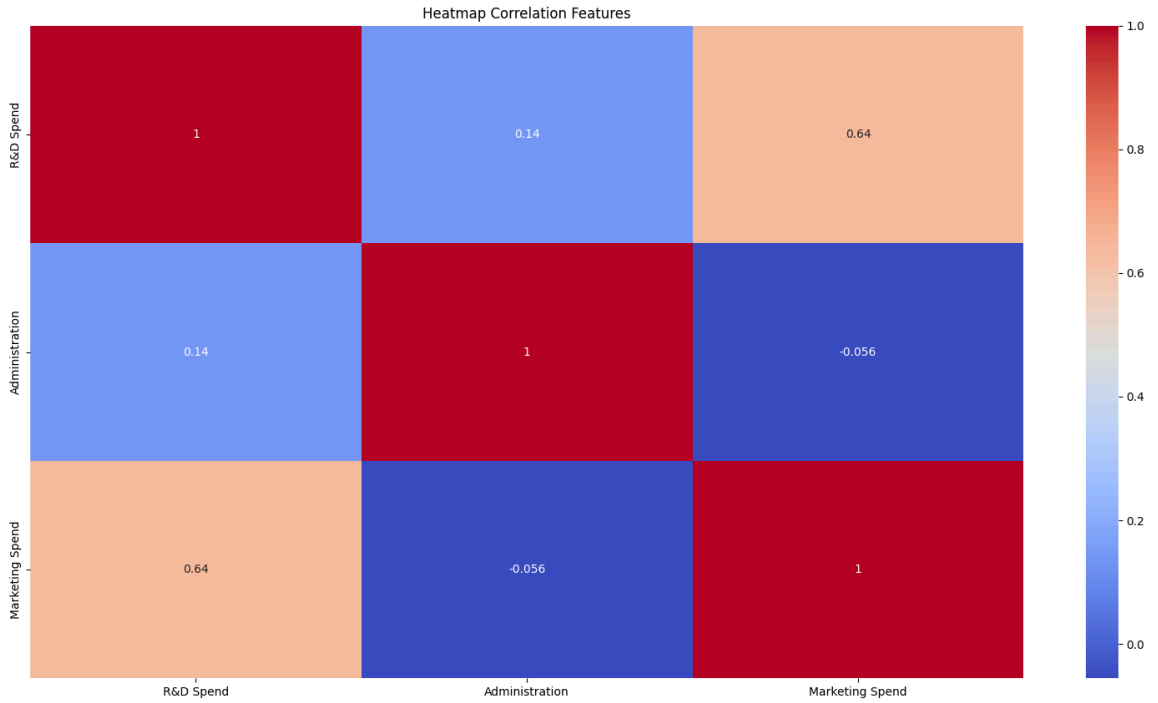
Câu 1.1

```
# Đọc dữ liệu
df = pd.read_csv('50_Startups.csv')

# Cleaning data: Fill missing bằng mean
numeric_cols = ['R&D Spend', 'Administration', 'Marketing Spend']

# Show trước khi fill missing
df_display_before = df.copy()
df_display_before[numeric_cols] = df_display_before[numeric_cols].fillna(0)
print("Trước khi fill missing values (NaN hiển thị dưới dạng 0.0):")
print(df_display_before.head(10).to_string(index=True))

# Fill missing (fill bằng mean)
for col in numeric_cols:
    if df[col].isnull().sum() > 0:
        mean_val = df[col].mean()
        df[col] = df[col].fillna(mean_val)
        print(f"Đã fill missing ở cột '{col}' bằng mean: {mean_val}")

# Show sau khi fill missing
print("\nSau khi fill missing values:")
print(df.head(10).to_string(index=True))
```

```
Trước khi fill missing values (NaN hiển thị dưới dạng 0.0):
   R&D Spend  Administration  Marketing Spend       State     Profit
0  165349.20       136897.80        471784.10    New York  192261.83
1       0.00       151377.59        443898.53  California  191792.06
2  153441.51       101145.55        407934.54     Florida  191050.39
3  144372.41       118671.85        383199.62    New York  182901.99
4  142107.34        91391.77        366168.42     Florida  166187.94
5  131876.90        99814.71        362861.36    New York  156991.12
6  134615.46       147198.87        127716.82  California  156122.51
7  130298.13       145530.06        323876.68     Florida  155752.60
8  120542.52       148718.95        311613.29    New York  152211.77
9  123334.88       108679.17        304981.62  California  149759.96
Đã fill missing ở cột 'R&D Spend' bằng mean: 72084.87958333333
Đã fill missing ở cột 'Administration' bằng mean: 123179.8540425532
Đã fill missing ở cột 'Marketing Spend' bằng mean: 214887.37291666667

Sau khi fill missing values:
       R&D Spend  Administration  Marketing Spend       State     Profit
0  165349.200000       136897.80        471784.10    New York  192261.83
1   72084.879583       151377.59        443898.53  California  191792.06
2  153441.510000       101145.55        407934.54     Florida  191050.39
3  144372.410000       118671.85        383199.62    New York  182901.99
4  142107.340000        91391.77        366168.42     Florida  166187.94
5  131876.900000        99814.71        362861.36    New York  156991.12
6  134615.460000       147198.87        127716.82  California  156122.51
7  130298.130000       145530.06        323876.68     Florida  155752.60
8  120542.520000       148718.95        311613.29    New York  152211.77
9  123334.880000       108679.17        304981.62  California  149759.96
```

Câu 1.2

```python
# Visualization
corr_matrix = df[numeric_cols].corr()
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.title('Heatmap Correlation Features')
plt.show()
```


Heatmap Correlation Features
R&D Spend
Administration
Marketing Spend
1
0.14
0.64
0.14
1
-0.056
0.64
-0.056
1
R&D Spend
Administration
Marketing Spend
1.0
0.8
0.6
0.4
0.2
0.0

Câu 1.3

```python
mean = df[numeric_cols].mean()
print("\nMean:")
print(mean)

modes = df[numeric_cols].mode().iloc[0]
print("\nMode:")
print(modes)

print("\nVariance:")
print(df[numeric_cols].var())

print("\nMedian:")
print(df[numeric_cols].median())
```

```
Mean:
R&D Spend          72084.879583
Administration     123179.854043
Marketing Spend    214887.372917
dtype: float64

Mode:
R&D Spend              0.000000
Administration     123179.854043
Marketing Spend        0.000000
Name: 0, dtype: float64

Variance:
R&D Spend          1.941019e+09
Administration     7.055606e+08
Marketing Spend    1.430980e+10
dtype: float64

Median:
R&D Spend          72096.239792
Administration     123179.854043
Marketing Spend    214887.372917
dtype: float64
```

Câu 1*

```python
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

rmse = mean_squared_error(y_test, y_pred, squared=False)
print(f"\nRoot Mean Squared Error (RMSE): {rmse:.4f}")
```

```
Root Mean Squared Error (RMSE): 12082.9127
```

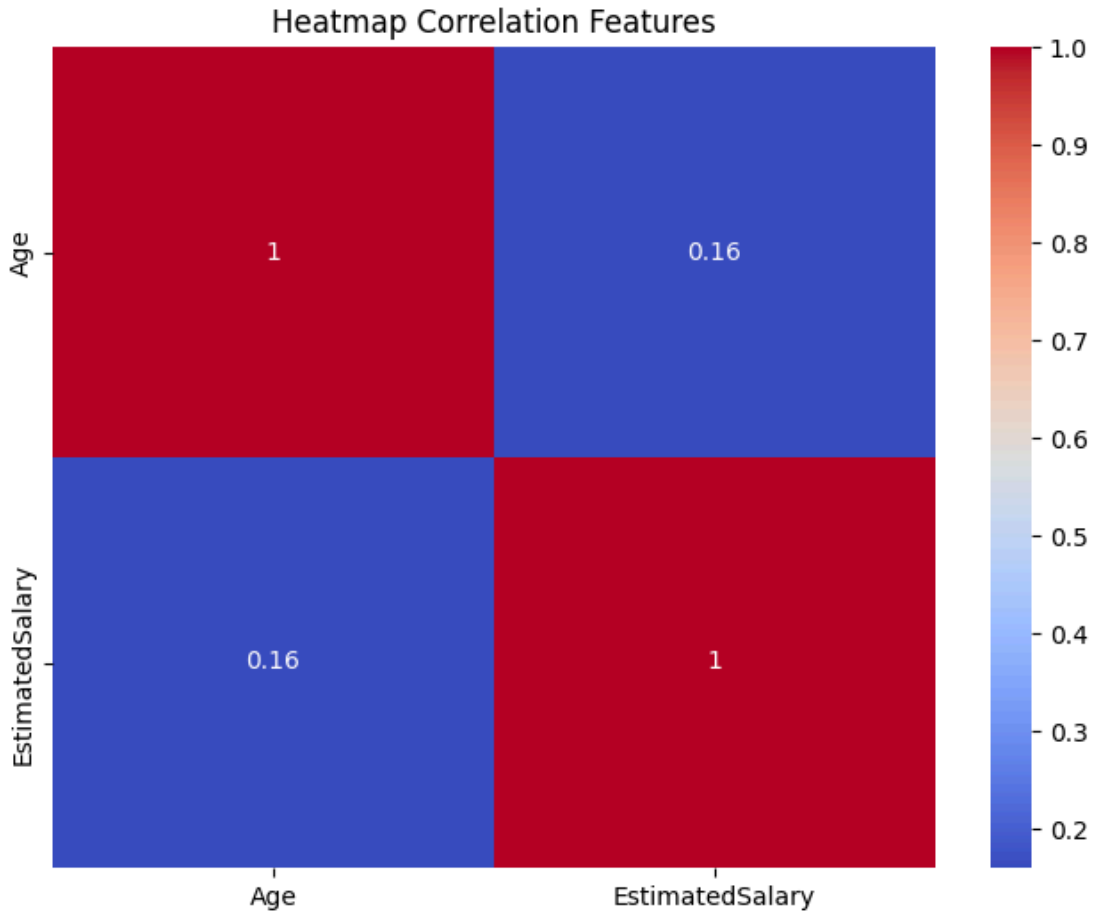
Câu 2.1:

```python
# Đọc dữ liệu
df = pd.read_csv('Social_Network_Ads.csv')

# Cleaning data: Fill missing bằng mean
numeric_cols = ['Age', 'EstimatedSalary']  # Specific features (không target)

# Show trước khi fill missing
df_display_before = df.copy()
df_display_before[numeric_cols] = df_display_before[numeric_cols].fillna(0)
print("Trước khi fill missing values: ")
print(df_display_before.head(10).to_string(index=True))

# Fill missing (fill bằng mean)
for col in numeric_cols:
    if df[col].isnull().sum() > 0:
        mean_val = df[col].mean()
        df[col] = df[col].fillna(mean_val)
        print(f"Đã fill missing ở cột '{col}' bằng mean: {mean_val}")

# Show sau khi fill missing
print("\nSau khi fill missing values:")
print(df.head(10).to_string(index=True))
```

```
Trước khi fill missing values:
    User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510    Male   19          19000.0          0
1  15810944    Male   35          20000.0          0
2  15668575  Female   26          43000.0          0
3  15603246  Female   27          57000.0          0
4  15804002    Male   19          76000.0          0
5  15728773    Male   27          58000.0          0
6  15598044  Female   27          84000.0          0
7  15694829  Female   32         150000.0          1
8  15600575    Male   25          33000.0          0
9  15727311  Female   35              0.0          0
Đã fill missing ở cột 'EstimatedSalary' bằng mean: 69916.44908616188

Sau khi fill missing values:
    User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510    Male   19     19000.000000          0
1  15810944    Male   35     20000.000000          0
2  15668575  Female   26     43000.000000          0
3  15603246  Female   27     57000.000000          0
4  15804002    Male   19     76000.000000          0
5  15728773    Male   27     58000.000000          0
6  15598044  Female   27     84000.000000          0
7  15694829  Female   32    150000.000000          1
8  15600575    Male   25     33000.000000          0
9  15727311  Female   35     69916.449086          0
```

Câu 2.2

```
# Visualization
corr_matrix = df[numeric_cols].corr()
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.title('Heatmap Correlation Features')
plt.show()
```



Câu 2.3

```
print("\nThống kê cơ bản trước khi fill missing:")
for col in numeric_cols:
    print(f"\n--- Cột {col} ---")
    print(f"Mean     : {df[col].mean()}")
    print(f"Mode     : {df[col].mode().values[0]}")
    print(f"Median   : {df[col].median()}")
    print(f"Variance : {df[col].var()}")
```

```
--- Cột User ID ---
Mean     : 15691539.7575
Mode     : 15566689
Median   : 15694341.5
Variance : 5134915051.833277

--- Cột Age ---
Mean     : 37.655
Mode     : 35
Median   : 37.0
Variance : 109.89070175438596

--- Cột EstimatedSalary ---
Mean     : 69916.44908616188
Mode     : 69916.44908616188
Median   : 69916.44908616188
Variance : 1127717609.9517722

--- Cột Purchased ---
Mean     : 0.3575
Mode     : 0
Median   : 0.0
Variance : 0.230269423558897 23
```

Câu 2.4:

```python
# Normalization: Min-Max cho specific numeric features
scaler = MinMaxScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

# Bỏ cột 'Gender' vì không cần one-hot encode
df = df.drop('Gender', axis=1)

print("\nDữ liệu sau normalization (features):")
print(df.head())
```

```
Dữ liệu sau normalization (features):
    User ID       Age  EstimatedSalary  Purchased
0  15624510  0.023810         0.029630          0
1  15810944  0.404762         0.037037          0
2  15668575  0.190476         0.207407          0
3  15603246  0.214286         0.311111          0
4  15804002  0.023810         0.451852          0
```

Câu 2*

```python
# Logistic Regression: Features = normalized numeric (không có Gender), Target = Purchased
feature_cols = numeric_cols
X = df[feature_cols]
y = df['Purchased']  # Target binary

#Scatter data
plt.figure(figsize=(8, 6))
colors = ['blue' if val == 0 else 'red' for val in y]
plt.scatter(X['Age'], X['EstimatedSalary'], c=colors, alpha=0.7)
plt.xlabel('Age (normalized)')
plt.ylabel('EstimatedSalary (normalized)')
plt.title('Scatter Plot Data for Logistic Regression')
plt.grid(True)
plt.show()

#Fit và Visualization
model = LogisticRegression()
model.fit(X, y)
y_pred_all = model.predict(X)

#Confusion matrix cho full data (visual)
cm_all = confusion_matrix(y, y_pred_all)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_all, annot=True, fmt='d', cmap='coolwarm')
plt.title('Confusion Matrix (Full Data)')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

```python
# Train/test split, predict, F1-Score + Confusion Matrix
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

f1 = f1_score(y_test, y_pred)
print(f"\nF1-Score: {f1:.4f}")

# Confusion matrix cho test set
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix (Test Set):")
print(cm)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='coolwarm')
plt.title('Confusion Matrix (Test Set)')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

```
F1-Score: 0.7917

Confusion Matrix (Test Set):
[[51  1]
 [ 9 19]]
```